Function cross-references for pgrust

Building a who-calls-whom index over the Rust code in *crates/*: what to use, and what has to be built

Recommendation. Don't use LXR, and don't write an analyzer from scratch. Use rust-analyzer's SCIP export for precise definitions and references, and add one small custom tool that turns those into caller/callee edges, follows pgrust's `seam!` indirection, and renders static pages. Start with `crates/backend/storage`, then run it on the whole workspace.

1. What the codebase looks like

Measured at pgrust v0.4-beta, commit 38634b4, excluding the vendored C reference in `crates/postgres-18.6-reference`:

Crates	Rust files	Lines of Rust	seam! declarations
938	6,723	≈ 3.0 million	≈ 1,540

Crates call each other through **seams**: function-pointer slots declared with `seam_core::seam!` and filled in at startup by the crate that owns the code:

```
// caller: crates/backend/access/heap/heaptoast/src/worker.rs:417
smgr_seams::smgr_nblocks::call(heap, ForkNumber::MAIN_FORKNUM)?;

// implementation plugged in at boot: crates/backend/storage/smgr/smgr/src/lib.rs:928
smgr_seams::smgr_nblocks::set(|rlocator, forknum| { ... });
```

Every standard tool (LXR, Sourcegraph, rust-analyzer, LLVM call graphs) shows the caller reaching `smgr_nblocks::call` and stops there, because the real target is a pointer set at runtime. Many of the cross-subsystem calls, which are the interesting ones, go through seams. **A tool that understands seams is the part that has to be built.**

2. Why not LXR

LXR and Bootlin's Elixir (its modern descendant) match identifiers as text and were designed for C. They answer “where does the name smgr_nblocks appear”, not “which function calls which”. They can't resolve Rust methods, traits or macros, and they don't produce a call graph at all.

3. Recommended pipeline

1. **Precise definitions and references.** `rust-analyzer scip .` exports a SCIP index: every definition and reference, fully resolved, across all crates and through macros. Expect it to take a while and need a lot of RAM on 3 million lines.
2. **Caller → callee edges.** For each reference, find the function whose body encloses it. Get function body ranges by parsing the files with the `syn` crate.

3. **Connecting the seams.** Detect `X_seams::f::call(…)` as an edge into the seam, and the non-test `X_seams::f::set(…)` as an edge from the seam to its implementation (following the closure or function path). That turns about 1,500 dead ends into real edges. Skip `set` calls in `tests.rs`, which plug in fakes.
4. **Output.** One static page per function listing its callers and callees, linked to GitHub at the pinned commit, optionally with a small Graphviz diagram of its immediate neighbours. With tens of thousands of functions, generate this as its own static site, or as JSON plus one search page, rather than through the Zola blog.

4. Limitations to expect

- A call to a trait method resolves to the trait's declaration, not the specific `impl`.
- `dyn Trait` calls can't be resolved statically.
- Getting those exactly right would mean analysing compiled MIR, which is much more work. For understanding the architecture, the pipeline above covers most of what's needed.

5. Off-the-shelf options worth knowing about

Option	Good for	Gaps
rust-analyzer call hierarchy (editor)	Reading interactively now: incoming and outgoing calls, one function at a time. VS Code exposes it directly.	Not a browsable site; stops at seams.
Sourcegraph + uploaded SCIP	Hosted web UI with precise go-to-definition and find-references.	No call trees; stops at seams.
Mozilla Searchfox	Full cross-reference site; supports Rust through SCIP.	Heavy to set up; stops at seams.
LXR / Elixir	Text-based identifier search for C.	No Rust semantics, no call graph.

6. Next step

Build a prototype of the pipeline as a separate Rust tool, scoped to `crates/backend/storage` first. That way the seam-stitching can be checked by hand against code already being read, before running it on all 938 crates.